

Projet : le jeu mario 2D guide

Les prérequis nécessaires pour la réalisation de ce projet sont : entrée/sortie, condition, boucle, fonction, pointeur, tableau (à 2 dimensions), structure, fichier, gestion d'erreur, allocation dynamique, librairie SDL et Github.

Si vous souhaitez travailler depuis la maison sur ce projet, il vous faudra installer sdl2 et sdl2-image. Je vous conseille fortement de l'installer sur une distribution linux et non windows, c'est beaucoup plus simple. Si vous n'avez pas linux sur votre ordinateur, vous pouvez soit utiliser une machine virtuelle, soit utiliser le sous système linux pour Windows (WSL). Pour ce faire, ouvrez l'application, exécutez et écrivez "optionalfeatures". Cochez la case Sous-système Windows pour Linux, puis faites OK, suivez ensuite les fenêtres qui s'affichent et redémarrez le PC. Une fois que vous avez WSL; vous pouvez installer une distribution Ubuntu via le microsoft store. Vous pouvez alors ouvrir une application Ubuntu (avec le numéro de la version) LTS qui vous donnera accès à une console Linux classique (sans interface graphique). Vous pouvez aussi remarquer un nouveau dossier sur votre PC qui contiendra l'ensemble des dossiers de cette "partition" Linux. Depuis la console Linux, écrivez les 2 lignes de commande suivantes :

- `sudo apt-get install libsdl2-dev`
- `sudo apt-get install libsdl2-image-dev`

Voilà vous avez installé les 2 librairies nécessaire pour lancer le projet (bien plus facile que sur windows). Vous pouvez alors facilement compiler votre projet via les commandes classiques sur la console Linux, si vous voulez un peu plus de confort vous pouvez installer sur votre windows l'IDE suivant : visual studio code. Il vous permettra de coder depuis votre windows, et de sauvegarder les différents fichier sur votre partie Linux pour effectuer la compilation. Vous devrez juste installer les extensions suivantes (partie droite du logiciel, ou raccourcie : Ctrl + shift + X): C/C++ et remote development.

Il vous suffira alors d'appuyer sur la petite icone verte en bas à gauche (avec les 2 chevrons blancs). Vous aurez alors une barre de recherche qui s'ouvre, sélectionnez "New WSL Windows using distro..." puis sélectionnez la distribution de Linux que vous avez installée. Vous pouvez alors facilement ouvrir un dossier à un endroit de votre "arborescence" Linux et y créer votre projet. Vous devrez alors simplement positionner votre console Linux au bon endroit et compiler/lancer votre code.

Si vous souhaitez quand même tout faire sur windows (même si la solution que je vous propose est assez proche), je vous conseille de regarder plusieurs tutoriels sur l'installation de la bibliothèque SDL2 (et pas 1) et de comment l'utiliser avec un IDE.

Activite 1

Nous allons réaliser un jeu en 2D, le premier mario bros. Le but sera de réaliser un jeu en 2 dimensions en utilisant la librairie SDL, l'ensemble du code sera en C. L'objectif final sera un jeu où l'on peut diriger notre personnage dans les 2 directions (+saut), à travers un niveau. Plusieurs améliorations seront proposées pour ajouter des options qui amélioreront le gameplay. Vous trouverez ci dessous un exemple de jeu (voir figure 1).



Figure 1: Jeu mario 2D

Vous allez réaliser ce jeu en binôme et, pour partager votre code, vous utiliserez un logiciel de versioning qui est Github. Vous devrez gérer vous même les moments où vous effectuerez vos commits et vous assurer que le code que vous envoyez à votre binôme est bien fonctionnel.

Vous aurez à disposition l'ensemble des sprites (mario avec les diverses directions, goomba, sol, ciel, bloc, tuyau, drapeau, bloc jaune), des fichiers qui vont représenter les niveaux et plusieurs fichiers représentant une architecture du logiciel.

Pour un projet plus important (comme celui-ci), il est important de diviser le code en plusieurs fichiers qui regrouperont des fonctions sur un thème similaire. Ici, nous aurons les fichiers suivants :

- `main.c` : contient la page d'introduction et permet de lancer notre jeu, c'est le fichier qui va contenir la fonction `main` (et le seul fichier qui aura cette méthode). Vous pouvez voir la page d'introduction sur la figure 2 (il s'agit d'une simple image).
- `game.c` et `game.h` : contiennent le cœur du jeu, notamment la boucle qui va appeler toutes les fonctions pour faire tourner le jeu. Toutes les méthodes en question seront dans d'autres fichiers.
- `file.c` et `file.h` : contiennent les fonctions qui vont charger les fichiers d'images et map. L'affichage de la map sera aussi géré dans cette méthode.
- `caractere.c` et `caractere.h` : contiennent la création des personnages (mario et éventuellement les ennemis) et leur affichage.
- `event.c` et `event.h` : gèrent l'ensemble des événements du jeu, c'est à dire les mouvements, le scroll de la map, les collisions (avec les obstacles, les ennemis), la sortie du décor et la fin du jeu (perdre ou gagner).

PS: tous les fichiers `.c` hormis le `main.c`, auront un fichier du même nom mais en `.h` qui sera le fichier de header. Ce fichier contiendra les prototypes des fonctions des fichiers `.c`, les différentes structures nécessaires pour notre fichier et éventuellement les include qui sont

nécessaires. Pour compiler cet ensemble de fichiers, vous aurez besoin de la ligne de commande suivante : `gcc main.c file.c game.c event.c caractere.c -o mario -lSDL2main -lSDL2_image -lSDL2`. Le début représente l'ensemble des fichiers que vous allez compiler (pas besoin de compiler tous les fichiers si vous ne les utilisez pas encore) et la fin représente l'ensemble des librairies nécessaires pour le projet.

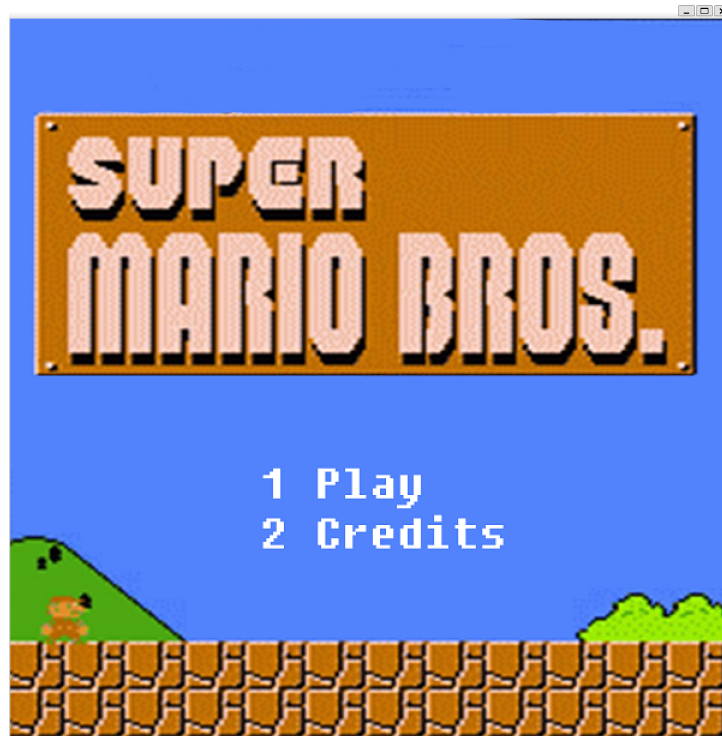


Figure 2: Fenêtre d'intro de mario

Pour vous aider vous aurez accès à différentes structures qui vous seront fournis pour vous aider lors de la programmation des différentes parties.

Attention, un soin particulier devra être mis en place sur la rédaction des commentaires dans votre code. En effet, vous allez coder à 2 personnes sur un même projet, votre collègue ne sera pas forcément au courant de ce que vous avez voulu faire avec un algorithme ou une certaines fonction, donc commentez votre pour votre et votre binôme. De plus il n'existe pas seulement qu'une seule bonne solution (il existe énormément de solution possible), donc ne vous arrêtez pas à ce que pourrez avoir codé quelqu'un de la classe. Finalement, un conseil très important pour la stabilité de votre code, à chaque fois que vous utilisez un malloc (pour instancier une structure, ...), il doit y avoir à la fin de votre code un free pour libérer la mémoire, de même pour la création de texture et sa libération avec `SDL_DestroyTexture` et pour le `render/window`. Si vous oubliez de faire le ménage à la fin de votre code, vous risquez d'avoir des bugs très particulier et incompréhensible.

Pour commencez vous allez travailler ensemble sur l'écran d'introduction et la gestion du dépôt github en parallèle.

(a) Mettez en place le dépôt sur Github en le partageant entre vos 2 comptes. Effectuez le premier dépôt sur celui ci pour l'initialiser.

(b) Dans le fichier `main.c`, réalisez un code permettant de générer une fenêtre vide de taille 1000*1000 (utilisez les constantes `LARGEUR_FENETRE` et `HAUTEUR_FENETRE` de `file.h`).

(c) Rajoutez l'image "menu.jpg" qui symbolisera notre image d'introduction (voir figure 2). Ne pas oublier de détruire la texture à la fin du programme (pour l'instant, vous pouvez mettre un `SDL_Delay` pour vérifier le bon fonctionnement).

(d) Récupérez les événements du clavier pour entrer dans le jeu. Vous devrez permettre de quitter la fenêtre si l'on clique sur la croix (bouton en haut à droite de la fenêtre) ou avec la touche `echap` (n'oubliez pas de nettoyer le tout en quittant). Vous devez aussi permettre de lancer le jeu si l'on appuie sur la touche 1. Pour ce faire, lors de l'appui sur la touche 1, vous lancerez la fonction `jouer(SDL_Render* renderer)` du fichier `game.c`.

(e) Finalement, ajoutez un écran de crédits simple avec le nom de votre binôme qui sera accessible lorsque l'on appuie sur la touche 2 depuis le home screen. Vous pouvez créer une image comme vous le souhaitez.

(f) Mettez en place les 2 branches dans le dépôt Github, une branche pour la partie Map et une autre branche pour la partie personnage. Vous fusionnerez ces 2 branches quand vous aurez chacun fini les 2 parties.

Les 2 activités suivantes correspondent aux tâches à effectuer par chacun des membres du binôme. Il faut bien avoir en tête que le travail effectué par vous deux est très connecté, et vous devrez discuter à plusieurs moments sur des points particuliers pour bien les appréhender. Si vous êtes bloqué à un endroit n'hésitez pas à regarder ce problème avec votre binôme. De même si vous avez fini votre parti, vous pouvez aider votre binôme sur la sienne pour pouvoir faire progresser le projet ensemble.

Activité 2

Cette partie se concentre sur la génération de la map et sur l'écran d'introduction.

Les questions qui suivent vont se réaliser dans le fichier `file.c` pour la gestion des fichiers de niveau, et dans le fichier `game.c` pour utiliser les différentes méthodes réalisées dans le fichier `file.c`. Avant de commencer, vous devez comprendre plusieurs choses. Tout d'abord, le niveau est stocké dans un fichier `niveau0.lvl` et `niveau1.lvl`. Vous utiliserez dans un premier temps le niveau 0 qui représente une plus petite fenêtre (et donc ne nécessite pas de scroll). Vous pourrez utiliser `niveau1.lvl` une fois que vous aurez mis en place la gestion du scroll.

Vous pouvez ouvrir les fichiers de niveau, il s'agit seulement de tableau à 2 dimensions qui contiennent des chiffres. La première ligne du fichier représente son nom, la deuxième le nombre de case de notre niveau (largeur hauteur) et ensuite l'ensemble des chiffres représentant le niveau. Chaque chiffre va représenter une case de notre map et chaque case sera de taille 33*33 pixel (utiliser la constante `Size_Sprite` de `file.h`). Il faudra charger les sprites (images utiliser pour notre map) correspondant aux chiffres en question :

- 0 : correspond au ciel et à l'image `sky.png`.
- 1 : correspond au sol et à l'image `sol.png`.
- 2 : correspond au bloc de pierre et à l'image `block.png`
- 3 : correspond à la boîte avec un point d'interrogation et à l'image `boite.png`
- 4 : correspond à une partie d'un tuyau et à l'image `tuyau1.png`
- 5 : correspond à une partie d'un tuyau et à l'image `tuyau2.png`

- 6 : correspond à une partie d'un tuyau et à l'image tuyau3.png
- 7 : correspond à une partie d'un tuyau et à l'image tuyau4.png
- 8 : correspond au poteau du drapeau qui sera utilisé en fin de niveau et à l'image fin1.png
- 9 : correspond au drapeau qui sera utilisé en fin de niveau et à l'image fin2.png
- 10 : correspond au lieu d'apparition des goombas, ils apparaîtront à cet endroit et bougeront pour le reste de la partie à partir de ce point.

Les images fournies ne sont pas définitives et, si vous souhaitez les améliorer, vous êtes libre de réaliser/récupérer d'autres images. Essayez juste de rester à la même taille d'image (40*40 pixels) pour éviter les déformations et modifiez le code pour l'adapter.

Pour finir, vous aurez accès à 2 structures : Map et Sprites. La structure Map va contenir 5 champs : int width, height (qui vont contenir le nombre de carrés qu'il y aura dans notre niveau), int **loadedMap (qui va contenir les chiffres du niveau qui sont dans le fichier, il s'agit d'un tableau à 2 dimensions) et int xscroll, yscroll (va contenir de combien vous avez scrollé dans les 2 directions pour faire bouger la fenêtre tout en gardant le personnage dedans). Cette structure contient donc toutes les informations nécessaires à la gestion de notre map et nous aurons besoin que d'une seule instance de cette structure. La structure Sprites quant à elle, va contenir 2 champs : SDL.Texture* sprite (qui va contenir une image d'un sprite de notre map) et int traverser (qui va contenir l'information si ce sprite est traversable (0) ou non (1)). Vous aurez donc besoin d'une liste de taille 10 (vous pouvez utiliser la constante NbSprites de file.h) qui va contenir les 10 textures possibles pour notre map, une seule instance de cette variable sera nécessaire aussi.

Bien entendu, l'ensemble des champs et des valeurs que je donne ici ne sont qu'à titre indicatif. Si vous avez de meilleures idées de fonctionnement, vous pourrez les mettre en place si vous le souhaitez. De plus, vous n'êtes pas obligé (et c'est même conseillé) de ne pas remplir et gérer l'ensemble des champs de ces structures dès le début. Vous pouvez vous focaliser, dans un premier temps, sur certains champs, puis, à chaque fois que vous rajoutez une fonctionnalité, vous pourrez alors gérer un champ de plus.

(a) Programmez une fonction permettant de lire le fichier de niveau (niveau0.lv1) et de récupérer les informations de largeur et de hauteur de la map. Attention, vous devez sauter la première ligne (vous pouvez la lire sans la stocker).

(b) Continuez la fonction précédente pour créer une instance de notre structure Map et remplissez les champs width et height. Il vous est conseillé de créer une variable de cette structure avec un malloc, cela vous permettra de supprimer cette variable seulement à la fin du programme avec un free.

(c) Permettez à la fonction de lire le tableau à 2 dimensions du fichier et de remplir le champ loadedMap de notre variable de type Map. Vous allez devoir initialiser le tableau avec un malloc au préalable.

(d) Réalisez une fonction qui va remplir une liste de variables de type Sprites contenant l'ensemble des images de notre décor. Cette fonction devra ouvrir l'ensemble des fichiers images et préciser si chaque Sprites est traversant ou non. Il vous est conseillé de créer avec un malloc un tableau dynamique de pointeurs de Sprite. Indice : l'ordre de cette liste à son importance, suivez les chiffres correspondants sur les fichiers de niveau.

(e) Écrivez une fonction `libererMap(...)` permettant de faire le nettoyage en détruisant, à la fin du programme, l'ensemble des textures et de libérer la mémoire non nécessaire (la mémoire allouée par des `malloc`). Vous appellerez cette fonction à la fin de la fonction `jouer`.

(f) En reprenant un code similaire à celui dans le `main()`, ajoutez la possibilité de lire les événements clavier et notamment le bouton croix de notre fenêtre et la touche `echap` pour pouvoir quitter la boucle de notre programme.

(g) Utilisez les 3 fonctions que vous avez réalisées dans la fonction `jouer()` du fichier `game.c` pour créer notre Map et notre liste de Sprites. Vérifiez le contenu de notre Map avec un `printf`.

(h) Programmez une fonction `affichageMap` qui permettra d'afficher la map sur notre fenêtre, elle devra prendre en paramètre le `render`, la liste de Sprites et la Map. Ne prenez pas en compte le numéro 10 de la Map (le goomba) et affichez le comme étant le ciel (vous changerez ça plus tard quand vous mettrez en place le goomba). Indice : il vous suffit de récupérer le numéro sur la map de `loadedmap`, de l'utiliser dans la liste de sprite et de placer l'image au bon endroit.

Activite 3

Cette partie sera plus centrée sur la création de notre personnage mario, son mouvement et la gestion des obstacles. Pour ce faire, étant donné que vous n'aurez pas accès dans un premier temps à la map, vous allez utiliser un écran blanc simple dans lequel vous effectuerez vos tests pour vérifier la bonne gestion de vos textures et mouvements.

Les questions qui suivent vont se réaliser dans les fichiers `caractere.c` et `event.c` pour la gestion des personnages et des événements et dans le fichier `game.c` pour l'utilisation des diverses méthodes au sein de la méthode `jouer()`. Tout d'abord, il faut comprendre le fonctionnement et le déplacement de notre personnage (mario ici). Celui-ci peut être assimilé à un rectangle sur lequel on colle une image qu'on déplace dans les deux directions possibles (x et y). Ce rectangle va se déplacer dans la map qui sera gérée par votre binôme mais ne pourra pas rentrer dans des éléments non traversable (voir structure Map). Pour finir, vous aurez une structure a utilisé qui symbolisera notre personnage. Dans le fichier `caractere.h`, vous trouverez la structure `Personnage` qui contiendra 9 champs :

- `SDL_Texture **image` : une liste des images (de pointeur sur image pour être précis) qui contiendra l'ensemble des sprites de notre personnage (ici mario).
- `SDL_Rect position` : la position à l'instant t de notre personnage mario et plus particulièrement du carré qui le symbolise.
- `int jump` : symbolise si le personnage saute ou non, vaudra 1 à partir du moment où le personnage commence à sauter et vaudra 0 à partir du moment où le personnage redescend (ou si le personnage ne saute pas).
- `int jumptime` : correspond à la durée depuis que le personnage effectue un saut, jusqu'au moment où il atteint le point culminant de son saut. Après le point culminant atteint, sa valeur va descendre jusqu'à retomber sur le sol. Son utilité principale est de bien gérer le temps de saut en mettant en place un temps de saut maximum que vous devrez trouver vous même.
- `gravite` : vaut 1 si le personnage est dans les airs (et donc que la gravité doit s'appliquer), vaut 0 sinon. Ce champ est utilisé pour tomber s'il n'y a pas de sol en dessous du personnage.

- `int direction` et `int derniereDirection` : symbolisent la direction (ou la direction précédente) vers laquelle mario se dirige, vaut 1 pour la droite, 2 pour la gauche et 0 pour une absence de mouvement.
- `int temps` : doit être incrémenté de manière régulière et sera utilisé pour faire varier les sprites de mario pour donner une impression de mouvement.
- `int win` : symbolise l'état de la partie, vaut -1 si le personnage perd (tombe dans le vide par exemple), 1 s'il gagne en touchant le drapeau et 0 sinon.

Bien entendu, l'ensemble des champs et des valeurs que je donne ici ne sont qu'à titre indicatif. Si vous avez de meilleures idées de fonctionnement, vous pourrez les mettre en place si vous le souhaitez. De plus, vous n'êtes pas obligé (et c'est même conseillé) de ne pas remplir et gérer l'ensemble des champs de cette structure dès le début. Vous pouvez vous focaliser dans un premier temps sur certains champs, puis à chaque fois que vous rajoutez une fonctionnalité vous pourrez alors gérer un champ de plus.

(a) En utilisant la structure `Personnage` décrite ci dessus, réalisez une fonction permettant de créer une instance `Personnage` qui correspondra à Mario. Il vous est conseillé de créer une variable de cette structure avec un `malloc`. Remplissez les différents champs de notre structure avec des valeurs correspondant à la situation initiale (position initiale du personnage, direction, ...). Utilisez cette fonction (et toutes les suivantes) dans la fonction `jouer()` du fichier `game.c`.

(b) Rajoutez une fonction `freePersonnage()` permettant de libérer la mémoire des différentes allocations dynamiques effectuées sur ce personnage et de détruire les textures. Elle sera utilisée à la fin de la fonction `jouer()`.

(c) Écrivez une fonction permettant d'afficher notre personnage sur notre écran dans son rectangle de position, dans le fichier `charactere.c`, et affichez le résultat sur notre écran blanc (n'oubliez pas d'utiliser la fonction `freePersonnage()` en ajoutant un delay avant son emploi pour bien voir le résultat et bien gérer la mémoire). Vous ne prendrez pas en compte le changement de sprite pour l'instant, et vous utiliserez un seul sprite basique pour Mario (par exemple, l'image avec Mario statique vers la droite). Le code sera modifié plusieurs fois pour accommoder des fonctionnalités supplémentaires.

(d) Mettez en place une gameloop dans la fonction `jouer()` et récupérez les événements d'appui sur les 3 touchés du clavier : gauche, droite et haut (pour le saut). Modifiez la valeur des paramètres de votre variable personnage en fonction de la direction choisie. Vous devez séparer le mouvement du personnage de la récupération de l'événement "appui sur la touche clavier".

(e) Programmez une fonction de déplacement qui sera écrite dans le fichier `event.c`. Cette fonction permettra, dans un premier temps, d'effectuer le déplacement à droite et à gauche de votre personnage sans changer les sprites de notre personnage pour l'instant. Vous testerez le tout sur votre écran blanc et ajusterez la vitesse de déplacement. Indice : il vous suffira de déplacer le rectangle du personnage qui sera alors rafraîchi sur l'écran par le biais de la fonction d'affichage du personnage.

(f) Mettez en place le saut pour le personnage en utilisant les 2 champs de notre structure (jump et jumptime). Pour l'instant, vous gérerez seulement le saut simple (et non la chute sans saut), c'est à dire que si le personnage saute d'un point A sans bouger, il doit revenir sur ce point A. N'oubliez pas d'incrémenter/décroémenter la valeur de jumptime à chaque déplacement. Testez la fonctionnalité pour trouver un bon équilibre pour la taille du saut et la vitesse de montée/descente.

(g) Complétez la fonction affichagePerso (éventuellement avec une fonction supplémentaire) pour changer les sprites en fonction du déplacement : droite, gauche, saut vers la droite et saut vers la gauche. Finalement, alternez entre les sprites pour les mouvements sans saut entre le sprite statique et le sprite de mouvement (ex: Mario1.png et Mario2.png) en utilisant le champ temps de notre personnage et en l'incrémentant au bon moment. Testez le résultat pour ajuster la vitesse d'alternance des sprites.

Activité 4

A partir de maintenant, vous aurez besoin du code des deux personnes pour effectuer les tâches suivantes. Vous pouvez néanmoins commencer à effectuer certaines de ces tâches. Dans le cas où vous avez tous les deux fini vos activités, vous pouvez prendre des tâches différentes pour cette activité. Une répartition des tâches possible serait de laisser une personne gérer le scrolling, tandis que l'autre personne gère les obstacles. Une bonne partie des tâches qui suivent sera réalisée dans le fichier event.c. Bien entendu, après que vous aurez réalisé chacune des tâches, vous devrez l'intégrer dans la méthode jouer() du fichier game.c et la tester.

(a) Effectuez l'intégration du code entre les deux activités précédentes pour pouvoir effectuer le mouvement du personnage dans la map (il va bien entendu traverser tous les blocs).

(b) Programmez une fonction permettant de détecter la collision entre notre personnage (plus particulièrement son rectangle) et tous les éléments du décor autour de lui. Le but sera d'analyser dans la variable map tous les éléments présents autour de la position de notre personnage, de vérifier leurs valeurs et donc à quel sprite elles correspondent pour finalement savoir si elles sont traversables ou non. En fonction, la fonction doit renvoyer 1 ou 0 pour préciser si le personnage est en collision avec un objet non traversable ou non.

(c) Mettez en place la méthode précédemment effectuée en modifiant la gestion du déplacement pour stocker les déplacements effectués dans une variable de type SDL_Rect temporaire. Vous éditez alors le rectangle de position de notre personnage seulement s'il n'y a pas de collision.

(d) Réalisez une fonction permettant de savoir s'il y a un sol ou non en dessous du personnage. Vous pouvez vous inspirer du fonctionnement de la fonction qui gère les collisions.

(e) Utilisez la méthode réalisée précédemment pour le champ "gravité" de notre variable Personnage afin de faire tomber le personnage au sol s'il n'y a pas de sol en dessous de lui.

(f) Programmez une fonction permettant d'empêcher mario de sortir de l'écran (à gauche, à droite et en haut seulement). Vous devrez calculer sa position par rapport à la map et vérifier qu'il n'en sorte pas. De la même manière que pour les fonctions précédentes, vous devrez intégrer cette fonction au sein du déplacement.

Le but des 3 questions qui suivent est de mettre en place un scroll pour la map, c'est à dire que lorsque notre personnage bouge dans une direction, l'affichage de la map doit le suivre. On va suivre la règle suivante: dès lors que le personnage atteint la moitié de l'écran en largeur (en partant de la gauche), l'écran doit suivre le mouvement du personnage de telle sorte que le personnage reste au milieu de la fenêtre. Il se passe la même chose si le personnage arrive à la moitié de l'écran en hauteur. Pour ce faire, nous n'allons pas changer la valeur de la position du personnage, mais modifier l'endroit où on affiche le personnage sur la map et faire défiler l'affichage de la map.

(g) Mettez en place une fonction qui va remplir les champs xscroll et yscroll de notre variable Map. Pour ce faire, vous devez considérer 2 cas: soit le personnage n'est pas assez loin sur l'écran, et dans ce cas, les 2 champs valent 0, sinon, ils doivent être égale de combien le personnage aurait bougé depuis le centre de l'écran.

(h) Modifiez la/les fonctions d'affichage de map pour prendre en compte ce scrolling. Ces modifications doivent permettre de faire défiler la map. Vous devrez agir sur la partie du champ LoadedMap de la variable map que vous allez lire.

(i) Modifiez la/les fonctions d'affichage de personnage pour prendre en compte ce scrolling. Ces modifications doivent permettre de fixer le personnage au centre de la map lorsque l'on scroll.

(j) Modifiez vos fonction pour changer la valeur du champ win de notre personnage à 1 si on gagne (on touche le drapeau) ou -1 si on perd (on tombe dans un trou).

(k) Programmez une fonction permettant de gérer la fin de niveau. Elle doit afficher les images win.png ou lose.png (vous pouvez les modifier si vous le souhaitez) dans la bonne situation. Vous devez alors permettre de quitter le jeu en appuyant sur la touche echap, ou de reprendre une partie en appuyant sur la touche 1.

Activite 5

Cette partie sera plus axée sur un ensemble de fonctionnalités additionnelles permettant une meilleure expérience de jeu.

(a) En utilisant la structure Personnage (mais pas besoin de l'ensemble des champs) et en vous inspirant des fonctions que vous avez utilisées pour mario, mettez en place l'affichage d'un goomba.

(b) Faites bouger les goombas de droite à gauche, en faisant changer la direction à chaque fois qu'ils percutent un mur.

(c) Programmez des fonctions pour gérer la collision avec les goombas. En cas de collision frontale avec un goomba, cela doit entraîner une défaite, et si le personnage saute dessus, mario doit rebondir et faire disparaître le goomba (sprite 3 du goomba).

(d) Rajoutez un chronomètre en haut à droite de l'écran, démarrant à 2 minutes et diminuant au fur et à mesure. Lorsque le chronomètre atteint 0, le joueur a perdu.

(e) Rajoutez un score juste en dessous du chronomètre. Le score sera calculé de la façon suivante (plusieurs points seront ajoutés plus tard) : casser un bloc rapporte 5 points, chaque pièce rapporte 15 points et tuer un goomba rapporte 10 points. A la fin du niveau, en fonction de la position ou mario touche le drapeau, il gagne plus ou moins de points (100 points tout en haut, 10 points en bas). Finalement, à la fin de la partie, le temps restant est converti en points (1 seconde = 1 point).

(f) Rajoutez des pièces dans les blocs avec un point d'interrogation. Lors d'un saut de mario sous un bloc avec un point d'interrogation, une pièce doit apparaître au dessus. Cette pièce sera immobile et doit disparaître après contact avec mario. Le nombre de points doit alors changer. Attention, mario ne peut pas réutiliser un bloc jaune qui a déjà été utilisé (indice : vous pouvez utiliser une deuxième variable tableau dans la structure map qui contiendra les cases utilisées).

(g) Mettez en place le système de champignons dans mario. Ils doivent apparaître de manière aléatoire sur un bloc (de la même façon qu'une pièce) avec une probabilité de 50% par rapport aux pièces. Lors de leurs apparitions, ils doivent partir dans une direction aléatoire et bouger de la même façon que les goombas (rebondir de droite à gauche sur les murs).

(h) Programmez la collision de mario avec un champignon. Lors de cette collision mario doit gagner une vie et changer de sprite d'affichage (voir l'image de mario non utilisée). La vie sera perdue lorsque mario touche un goomba et il aura de nouveau les sprites classiques. De plus mario peut casser les blocs en sautant en dessous d'eux dans cette forme (et gagner des points supplémentaires).

(i) Rajoutez des musiques dans le jeu (voir les fichiers musique fournis). Vous avez à disposition une musique d'ambiance, pour le saut, en cas de défaite/victoire, pour le champignon, pour casser un bloc, pour les pièces et lorsque vous tuez un goomba.

(j) Mettez en place d'autres ennemis comme les koopas.

(k) Proposez des améliorations pour le jeu.